

第十章 稀疏奖励

什么是稀疏奖励？例如，一个机械臂需要完成一个很复杂的任务才会得到奖励，而途中的几千个小步骤都没有奖励。（**0 奖励占绝大多数，而非 0 奖励很少。**）

对于一个什么都不会的智能体而言，几乎不可能学会这个动作，理由有二：

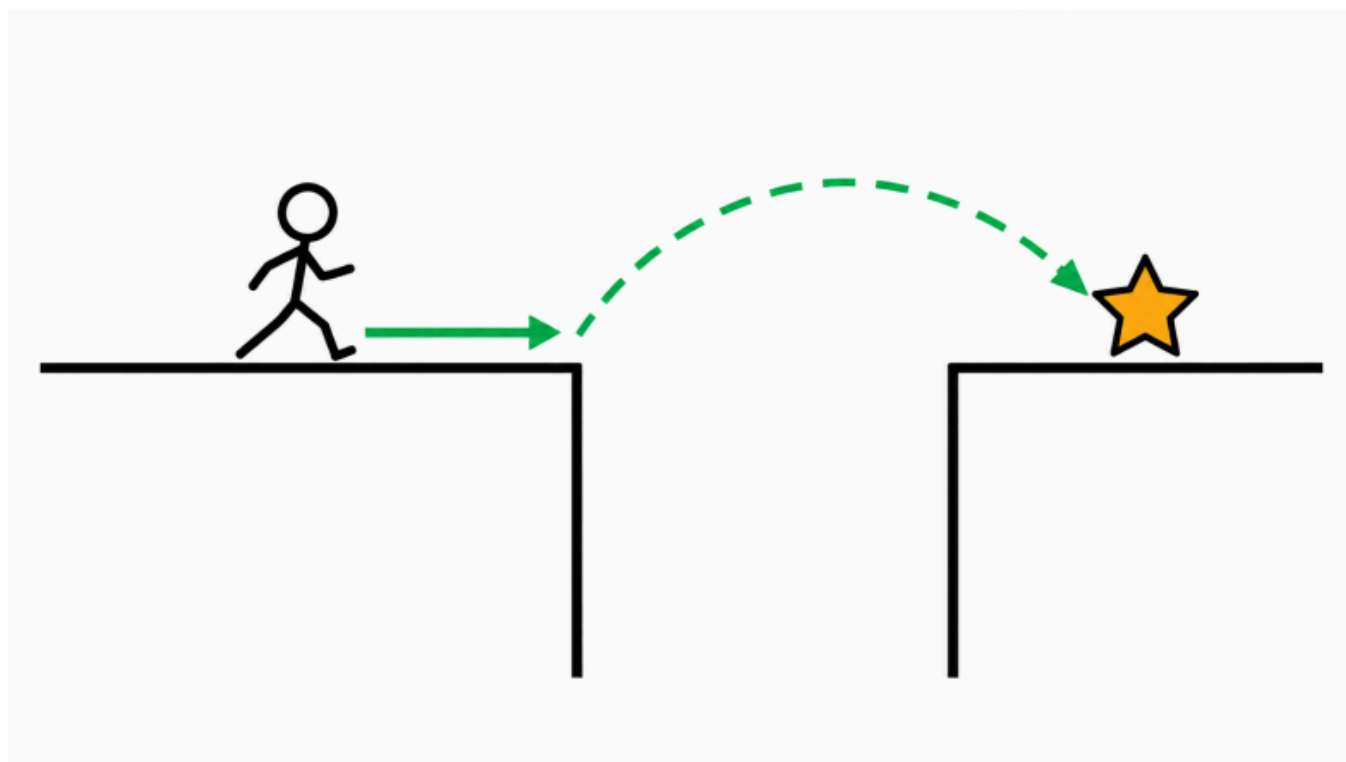
1. **探索困难**：从 0 开始随机探索，恰好几千步都走对、完成任务并获得奖励的概率几乎为 0。
2. **信用分配困难**：即使做对了，智能体也不知道“我之前做的那几千步动作到底对最后得到的这个奖励有什么贡献”。

为了解决稀疏奖励这个问题，我们提出了以下四个方案。

设计奖励

这个很好理解：稀疏奖励就是非 0 奖励太少，那我们就在过程中**人为加入一些奖励**，引导智能体更快学会相应动作。

例如，一个横版 2D 游戏中，智能体需要操控小人向右走，并跳跃越过一个深坑以后抵达终点，获得奖励。

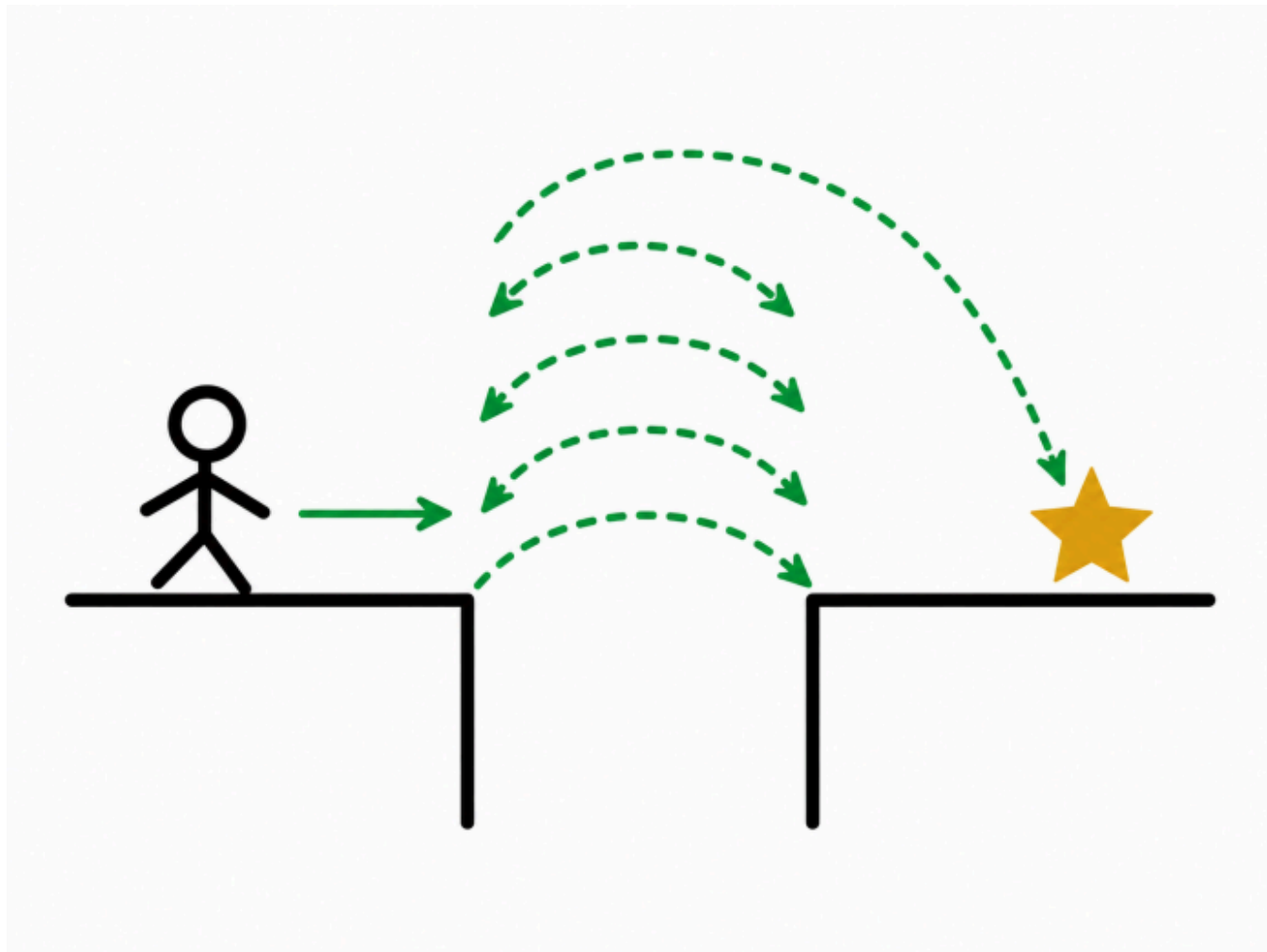


【图片：小人从左侧平台向右移动，并沿虚线轨迹越过深坑，到达右侧以星形表示的终点。】

如果不设计奖励，智能体只有在抵达终点的那一刻才能获得奖励，而这本身对于什么都不会的智能体来说几乎不可能。所以我们**设计一些辅助奖励**来引导智能体采取我们希望的动作，例如：智能体向右运动时给奖励，反方向走时给惩罚；**越过深坑而不掉进去时给奖励**，掉进深坑时给惩罚；过长时间没能完成任务时也给惩罚等。

但是这样也有弊端：如果**辅助奖励设计得不合理**，可能导致智能体“**走上歪路**”。

还是上面的例子，我们设计了一条“越过深坑而不掉进去时给奖励”，那么智能体可能会学到如下图所示的策略：通过“反复横跳”来**刷分**。



【图片：小人在深坑两侧沿多条往返虚线轨迹反复跳跃，而右侧仍标有星形终点。】

很显然，这已经**背离了我们最初的目标**——尽快到达终点；但因为奖励设计不合理，这样做反而能获得更高的奖励。

所以奖励设计的关键，**不是让奖励变多，而是让奖励与真正目标保持一致，避免有空子可钻。**

好奇心

在上面的奖励设计中，我们人类之所以能设置一些定向奖励，往往是因为我们很清楚地了解环境、知道怎样做得好，因而可以引导智能体向这个方向靠近。

然而，对于**复杂环境**，我们可能**不知道应该给哪些行为奖励**，也不可能把所有有价值的探索行为都写出来。所以我们提出好奇心：好奇心**鼓励智能体探索“实际情况和自己预测相差较大”的情况**。

为了实现以上目的并尽量避免问题，我们提出三个网络，按照以下思路走一遍。

首先，最基础的道理是：**想鼓励一个行为，就要给它增加奖励**。所以这里，我们在原始的、由环境返回的奖励（称为**外部奖励** r_t^e ）的基础上，增加额外的、基于好奇心机制的奖励（称为**内在奖励** r_t^i ）。也就是说，总奖励为 $r_t = r_t^e + r_t^i$ 。

奖励	回答的问题	作用
外部奖励	“任务完成得好不好？”	指导最终目标
内在奖励	“这个状态我是否还不了解？”	鼓励探索未知

内在奖励怎么产生、如何决定大小呢？这就引出第一个网络：**预测网络**。它输入当前状态 s_t 以及采取的动作 a_t ，输出下一状态的预测值 $\hat{s}_{t+1}$ ，然后和下一状态的真实值 s_{t+1} 比较。二者误差越大，表明智能体对于这一步的**认识不够（陌生）**，这一步的**内在奖励越大**。

这样会产生一个问题：陌生或难以预测的事物不一定与实际需要解决的问题有关。例如，我在落叶纷飞的道路上想往前走，如果只关注落叶这一难以预测其下一状态的事物，而不聚焦于与动作相关的内容，就会失去好奇心的意义。于是这里提出第二个网络：**特征提取器**。这个网络非常简单，就是输入一个状态 s_t ，输出提取特征后的状态 $\phi(s_t)$ 。 $\phi(s_t)$ 应当**保留与任务相关的信息而舍弃无关信息**。因而需要**对预测网络进行改进**：输入 $\phi(s_t)$ 和 a_t ，输出 $\hat{\phi}(s_{t+1})$ 的预测值，与 $\phi(s_{t+1})$ 的真实值作比较，产生内在奖励。

特征提取器如何训练呢？这里提出第三个网络：**逆向模型**。它输入 $\phi(s_t)$ 和 $\phi(s_{t+1})$ ，输出 t 时刻所做动作 a_t 的预测值 $\hat{a}_t$ ，再与真实动作 a_t 作比较，**根据这个误差训练逆向模型和特征提取器**。

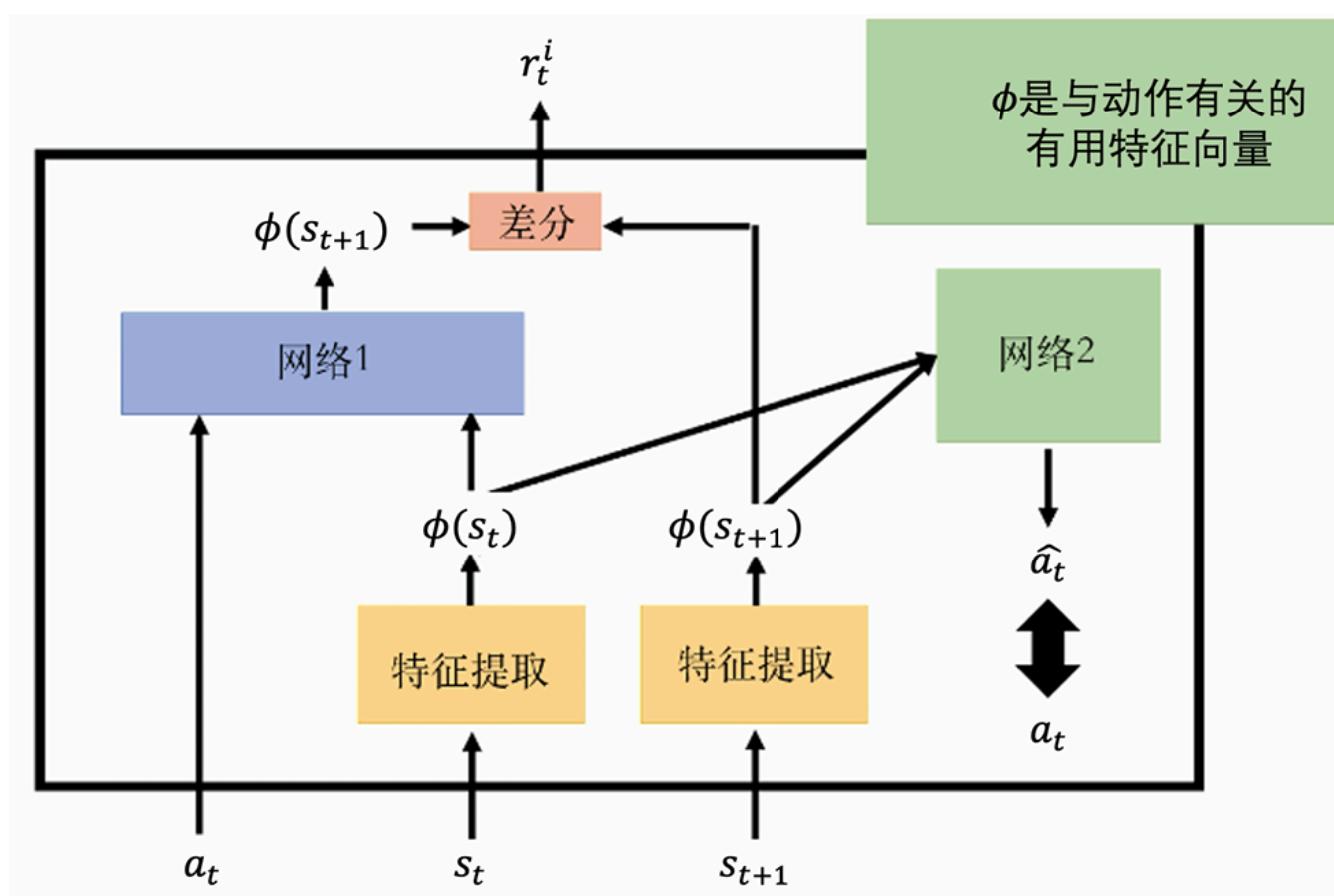
全流程：

- 智能体根据当前状态 s_t 选择动作 a_t 。
- 环境返回真实状态 s_{t+1} 以及外部奖励 r_t^e 。
- 特征提取器处理两个状态，得到 $\phi(s_t)$ 和 $\phi(s_{t+1})$ 。
- 预测网络根据 $\phi(s_t)$ 和 a_t 预测 $\hat{\phi}(s_{t+1})$ 。

5. 比较 $\phi(s_{t+1})$ 与 $\hat{\phi}(s_{t+1})$, 得到内在奖励 r_t^i 。
6. 用 $r_t = r_t^e + r_t^i$ 作为这一步的总奖励, **参与更新**。✱
7. 逆向模型根据 $\phi(s_t)$ 、 $\phi(s_{t+1})$ 预测 $\hat{a}_t$ 。
8. 比较 $\hat{a}_t$ 与真实动作 a_t , 用误差训练逆向模型和特征提取器。

✱ 好奇心不是一种独立的强化学习算法, 而更像是能够插入各种强化学习算法中的模块。从上面的流程也看得出来, 这个模块**对外暴露并修改的接口只有总奖励**这一个值, 也就是说, 好奇心模块接入各种强化学习算法时, 只是调整了奖励值。(当然, 也有可能和某个具体算法产生影响, 这需要逐个分析, 不能一概而论。)

下图是好奇心模块的图示, 可以看到, 输入就是正常的状态和动作, 输出就是修正后的奖励, 其他各种新网络和新内容都被封装在模块内部。



【图片: 状态 s_t 、 s_{t+1} 经特征提取得到 $\phi(s_t)$ 、 $\phi(s_{t+1})$; 网络 1 根据动作和当前特征预测下一特征, 二者差分产生内在奖励 r_t^i ; 网络 2 根据相邻状态特征预测动作 $\hat{a}_t$, 并与真实动作 a_t 比较。】

注意: 好奇心解决的是“智能体不愿意探索”的问题, 但不能单独解决“探索什么才真正有用”的问题。对于后者, 参见下面的表格。

问题	关注点	谁负责判断
什么变化很新奇？	以前是否见过、是否容易预测	前向预测模型
什么变化与动作有关？	状态变化能否解释智能体动作	逆向模型与特征提取器
什么变化有助于完成任务？	是否让智能体更接近最终目标	外部奖励、目标或任务评价

上面提到的特征提取器只从第一行进步到了第二行，但没有到达第三行。

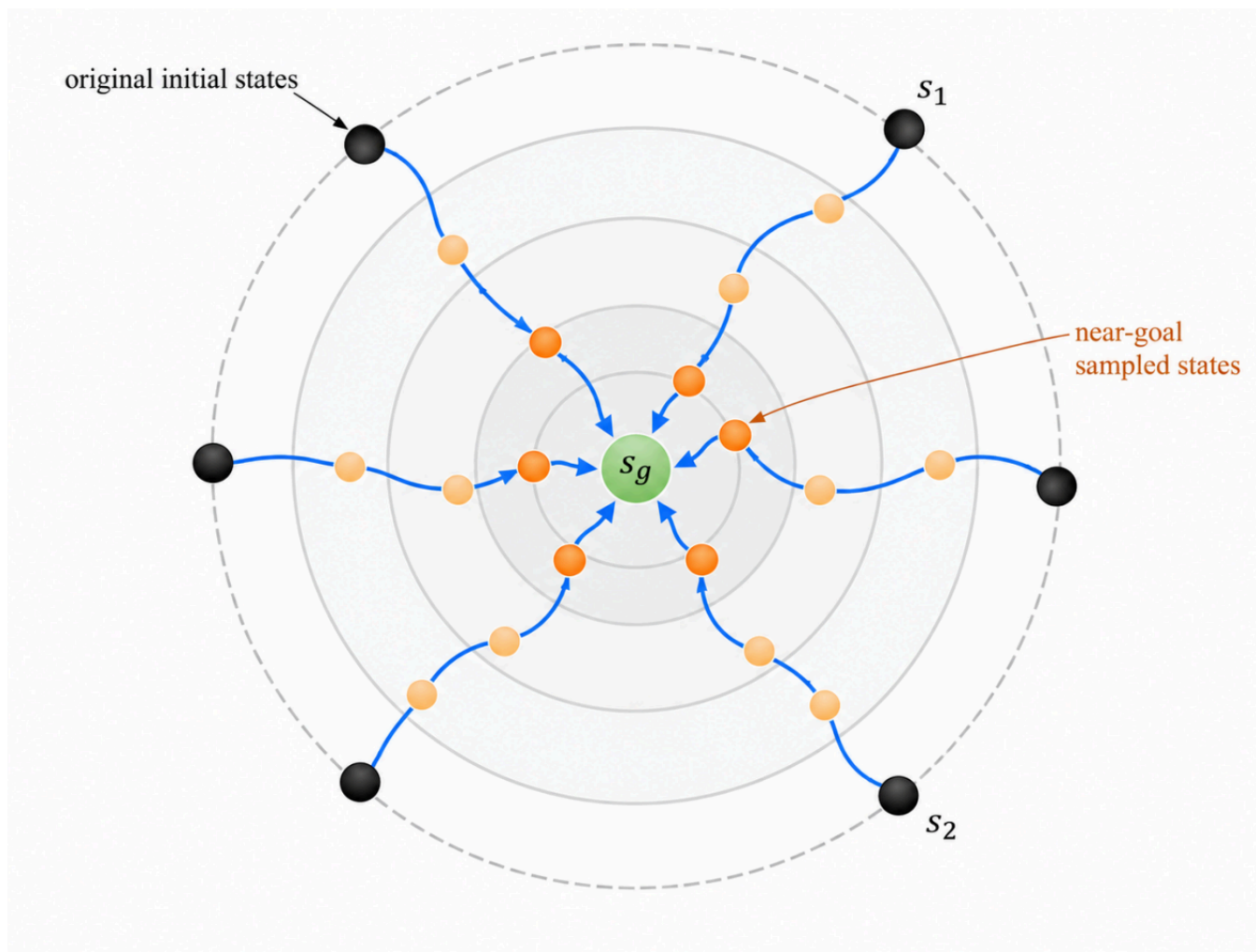
课程学习

“比如，假设我们要教一个小朋友学微积分，他做错题就惩罚他，这样他很难学会。我们应该先教他乘法，再教他微积分。”

课程学习有点像奖励设计，只不过这里不改变奖励，而是先**把困难任务缩减成简单任务**，让智能体“一步一步来”，由易到难，最终攻克原始的困难问题。

当然，从简单到中等再到困难的“课程”可以由人类来设计，但有一种更通用的方法：**逆向课程生成**。简单来说，把完成一个任务看成**从初状态 s_{start} 到末状态 s_{end} 的转移**，其流程如下：

1. **从未状态 s_{end} 出发**，在靠近末状态的状态中**采集若干状态**。
2. 让当前的智能体从采集的若干状态出发试跑几轮，查看转移到末状态的成功率。若成功率过低，说明当前状态转移到末状态这一任务对现在的智能体来说太难，不适合作为课程，所以舍弃；若成功率过高，说明任务太简单，学了没有意义，所以也舍弃；只有成功率适中，才说明该任务大致处于当前智能体的**能力边界**，适合作为课程，予以**保留**。假设保留了 s_1, s_2, s_3 。
3. 找到合适课程以后，**以这些状态 s_1, s_2, s_3 为起点**，训练智能体，使其能够逐渐解决从目前状态出发转移到末状态的问题。
4. **重复上述过程，但是将出发点改为 s_1, s_2, s_3** ，继续进行采集状态等循环。
5. 直到智能体的能力不断增强，最后采样**覆盖原始任务的状态分布并能够胜任**。



【图片：目标状态 s_g 位于中心，近目标采样状态和训练轨迹由内向外逐层扩展，最终覆盖外围的原始初始状态，包括 s_1 、 s_2 等。】

因为是从末状态一层一层“扩散”到初状态，像这样反着来，所以称为逆向课程生成。

分层强化学习

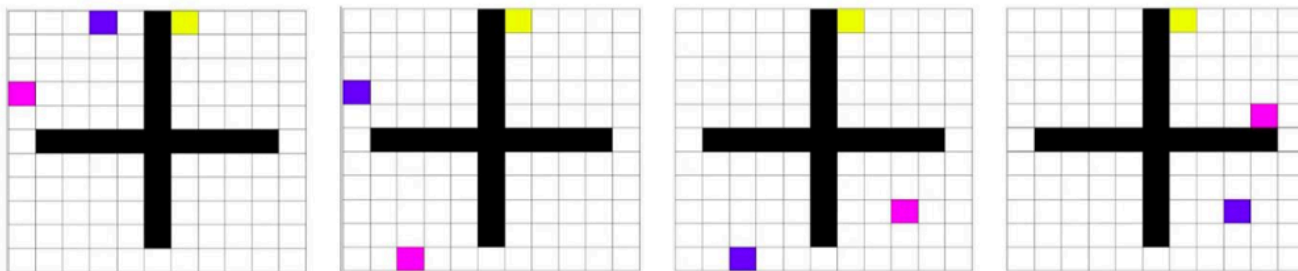
普通强化学习是：状态 → 直接选择底层动作 → → 最终奖励。

分层强化学习是：状态 → 高层选择子目标 → 低层执行动作 → 子目标完成 → 高层选择新目标。

高层回答：下一阶段要完成什么？

低层回答：具体应该怎么完成？

例如走迷宫：



【图片：四幅连续的网格迷宫示意图包含黑色十字形墙体、智能体位置、紫色子目标和黄色终点，展示智能体在不同阶段向子目标移动。】

蓝色是智能体当前位置，黄色是终点，黑色是墙。

高层告诉底层：去紫色的地方。底层只负责执行从蓝色到紫色。循环往复。

核心思想是：把一个漫长、稀疏奖励、难以探索的任务，拆成**多个由高层规划、低层执行的子任务**，让每个策略只负责自己擅长的决策层次。

这也和人类完成复杂任务的方式很像：我们通常**先制定计划，再执行局部动作**，而不是一开始就逐个决定几百个底层动作。